

Maze Solving Algorithm Comparison: BFS, DFS, A, and Ant Colony Optimization on Grid Mazes*

Noble Carpenter, Teagan Tobias, Christian Winchester

Computer Science Department

Tennessee Tech

Cookeville, Tennessee

jncarpente42@tntech.edu, ttobias42@tntech.edu, cwincheste43@tntech.edu

***Abstract*–This report compares four pathfinding algorithms – Breadth-First Search (BFS), Depth-First Search (DFS), A*, and Ant Colony Optimization (ACO) – on randomly generated 2D grid mazes of varying sizes and obstacle densities, using 20 trials per configuration. Performance was evaluated by execution time, memory usage, path length, nodes explored, and success rate. All source code, raw results, and graphs are available in the project’s public GitHub repository.**

I. PROBLEM DEFINITION

1.1 PROBLEM STATEMENT

The problem is shortest-path finding in an $N \times N$ 2D grid maze, where 0 represents open cells and 1 represents walls. Movement is limited to four directions with uniform cost, from start (0,0) to goal (N-1, N-1). The goal is to determine whether a path exists and, if so, return the shortest one. This is equivalent to unweighted graph traversal, where open cells are vertices and edges connect adjacent cells, giving $V \approx N^2$ (1-d) and $E \leq 4V$, so classical search runs in $\Theta(N^2)$ time.

1.2 RESEARCH QUESTIONS

Question I: How does peak memory usage scale with maze sizes for BFS, DFS, A*, and ACO?

Question II: How does obstacle density affect the number of nodes explored by each algorithm?

Question III: What is the trade-off between solution quality and execution time for ACO relative to the classical algorithms?

We expected BFS and A* to return optimal paths on every solvable instance. We expected A* to explore fewer nodes than BFS by virtue of its heuristic. We expected DFS to use less memory but return longer paths. We expected ACO to be slower and potentially non-optimal, but to converge toward near-optimal paths given sufficient iterations.

II. ALGORITHM DESCRIPTIONS AND THEORETICAL ANALYSIS

2.1 BREADTH-FIRST SEARCH – BENCHMARK

BFS explores the maze level by level using a first-in-first-out queue, visiting all nodes at distance k before $k+1$. A visited set prevents

re-expansion, and BFS guarantees the first time the goal is reached is along the shortest path.

- Always finds the optimal path
- Simple, deterministic baseline for comparison
- Memory-intensive due to large frontier sizes

Time Complexity: $\Theta(V + E)$

Space Complexity: $\Theta(V)$

2.2 DEPTH-FIRST SEARCH

DFS explores as deep as possible along a branch using a last-in-first-out stack, with randomized direction order for reproducibility. It stops when the goal is first found, regardless of path quality.

- Lower memory uses than BFS
- Often find a path quickly
- Does not guarantee shortest path; quality degrades with size

Time Complexity: $\Theta(V + E)$, worst case

Space Complexity: $\Theta(V)$, worst case

2.3 A* SEARCH

A* uses a priority queue with $f(n) = g(n) + h(n)$, where $g(n)$ is path cost and $h(n)$ is Manhattan distance. The heuristic is admissible and consistent, ensuring optimal solutions while guiding search toward the goal.

- Optimal and usually faster than BFS
- Expands fewer nodes using heuristic guidance
- Performance drops as obstacles increase and heuristic becomes less informative

Time Complexity: $\Theta(V + E)$, worst case

Space Complexity: $\Theta(V)$

2.4 ANT COLONY OPTIMIZATION – NATURE-INSPIRED ALGORITHM

ACO simulated ant foraging over 50 iterations with 20 ants per iteration. Paths are built probabilistically using pheromone levels and heuristic distance, with updates based on path quality and evaporation ($p = 0.5$). A step limit prevents infinite wandering.

- Bio-inspired, does not rely on explicit graph traversal
- Stochastic and parameter-dependent
- No guarantee of optimality and may fail to find a path in larger or denser mazes

Time Complexity: $\Theta(\text{iterations} \times \text{ants} \times N^2)$
(*~constant factor ~1000 x BFS*)

Space Complexity: $\Theta(N^2)$ for pheromone matrix

III. EXPERIMENTAL DESIGN

3.1 IMPLEMENTATION DETAILS

Programming Language: Python 3.x

Input Description: maze sizes range from 10x10 to 160x160 with obstacle densities from 10% to 40%. Each configuration is run for 20 independent trials using different random seeds, and results are averaged across runs.

Characteristics of Computer: standard desktop environment running Python (single-process execution with fixed system resources; no parallelization).

Reference Method: for optimality, BFS, which provides the true shortest path in unweighted grids, and all other algorithms were compared against it.

Performance Metrics:

Execution Time (s) – wall-clock time

Peak Memory (KB) – peak bytes allocated during algorithm execution

Path Length (steps) – number of moves in the returned path

Nodes Explored – total cells dequeued/expanded (BFS, A*), popped from stack (DFS), or cumulatively visited by all ants across all iterations(ACO)

Success Rate (%) – fractions of trials in which the algorithm returned a valid path to the goal

3.2 INPUT CONFIGURATIONS

Parameter	Values
Maze Sizes	10x10, 20x20, 40x40, 80x80, 160x160 (grid cells)
Obstacle Densities	10%, 20%, 30%, 40%
Trials per Configuration	20 (independent seeds per trial)
Total Experiment Runs	5 sizes x 4 densities x 4 algorithms x 20 trials = 1600 rows
Start / Goal	(0,0) → (N-1, N-1)
Seed Formula	(rows x 1000 + cols) x 100 +

	density x 100000 + trial
--	--------------------------

3.3 ACO PARAMETER SETTINGS

Parameter	Value	Rationale
Number of Ants	20	Sufficient colony diversity without excessive runtime
Iterations	50	Allows pheromone convergence on smaller grids
Pheromone Weight	1.0	Standard equal weighting
Heuristic Weight	2.0	Stronger goal-direction pull to help ants find paths
Evaporation Rate	0.5	Moderate evaporation; tested default
Pheromone Deposit	100.0	Scales deposit relative to path length
Max Steps per ant	NxN	Prevents infinite loops on unsolvable branches

IV. RESULTS

See annex, pages 1-5.

V. ANALYSIS AND INTERPRETATION

5.1 EMPIRICAL COMPLEXITY

VALIDATION

Empirical runtime growth from Table 1 is consistent with $\Theta(N^2)$ for BFS, DFS, and A*, as expected from theory. Going from 80x80 to 160x160, BFS runtime increases by a factor of 4.45 and DFS by 4.34, closely matching the linear-in-V prediction. A* scales slightly worse (5.96 factor). ACO runtime growth between larger sizes is a factor of 2.62, which is good.

5.2 TRADE-OFF SUMMARY

See annex, page 6.

5.3 LIMITATIONS OF EXPERIMENT

- Synthetic mazes only: all mazes randomly generated with uniform obstacle probability. Real world grids have structured obstacle patterns that could favor or disadvantage specific algorithms differently.
- Fixed start/goal: start at (0,0) and goal at (N-1, N-1) places the goal diagonally opposite the start. DFS with direction randomization may perform better or worse at other start/goal placements.
- ACO parameter sensitivity: the observed low success rate is likely highly sensitive to the choice of n_{ant} , $n_{\text{iterations}}$, and evaporation rate. A proper parameter search was outside the project timeline.

- Single machine measurement: absolute runtimes are machine-specific. Python GIL and OS scheduling may introduce timing noise.

5.4 WHAT WE'D DO DIFFERENTLY

- Increase ACO iteration and ant budget: 200+ iterations and 50+ ants with adaptive evaporation would likely raise the success rate significantly and produce more meaningful path quality data.
- Add a bidirectional BFS baseline: bidirectional BFS reduces the effective search radius from N to N/2, offering faster solution discovery without requiring a heuristic.
- Test structured mazes: perfect mazes (generated by recursive backtracking) have exactly one path between any two cells, eliminating the solvability-filtering step and isolating algorithm behavior on torturous paths.

5.5 FUTURE RESEARCH QUESTIONS

Question I: Can parallel ACO achieve competitive runtimes with significantly higher success rates at large grid sizes?

Question II: Would a bidirectional A* or jump-point search outperform standard A* on large, low-density grids?

Question III: How do these algorithms perform on non-uniform cost grids, where different terrain types carry different movement costs?

VI. REFERENCES

- [1] Chatgpt, chatgpt.com/. Accessed 16 Apr. 2026.
- [2] Claude, claude.ai/login. Accessed 16 Apr. 2026.
- [3] “Depth First Search or DFS for a Graph - Python.” *GeeksforGeeks*, GeeksforGeeks, 23 July 2025, www.geeksforgeeks.org/python/python-program-for-depth-first-search-or-dfs-for-a-graph/.
- [4] Dorigo, Marco, et al. *Ant Colony Optimization - Cs.Princeton*, 2006, www.cs.princeton.edu/courses/archive/fall24/cos597C/bib2/Ant_colony_optimization.pdf.
- [5] Editorial Professional Insights. “Ant Colony Optimization Python.” *Epsilon*, epsilon, epsilon.newpointe.org/resource/F4D/789/read_cc3X3w_AntColonyOptimizationPython. Accessed 16 Apr. 2026.
- [6] “Introduction to Ant Colony Optimization.” *GeeksforGeeks*, GeeksforGeeks, 13 Feb. 2026, www.geeksforgeeks.org/machine-learning/introduction-to-ant-colony-optimization/.
- [7] Kumar, Rajesh. *The A* Algorithm: A Complete Guide* | Datacamp, 2024, www.datacamp.com/tutorial/a-star-algorithm.
- [8] “Maze_dataset.” *Maze_dataset API Documentation*, understanding-search.github.io/maze-dataset/maze_dataset.html. Accessed 16 Apr. 2026.
- [9] McKee, Amberle. *Breadth-First Search in Python: A Guide with Examples* | Datacamp, 2024, www.datacamp.com/tutorial/breadth-first-search-in-python.
- [10] Various Authors. “5. the Import System.” *Python Documentation*, docs.python.org/3/reference/import.html. Accessed 15 Apr. 2026.
- [11] Various Authors. “Random - Generate Pseudo-Random Numbers.” *Python Documentation*, docs.python.org/3/library/random.html. Accessed 15 Apr. 2026.
- [12] Various Authors. “TRACEMALLOC - Trace Memory Allocations.” *Python Documentation*, docs.python.org/3/library/tracemalloc.html. Accessed 15 Apr. 2026.

VII. TEAM CONTRIBUTIONS

All three team members participated in brainstorming sessions to define the research questions, select algorithm parameters, and interpret results.

See annex for table distribution, page 6.

VIII. MAIN DIFFICULTIES AND ROADBLOCKS

The main challenge was ACO, whose stochastic behavior made debugging difficult compared to deterministic algorithms verified against BFS. Its unexpectedly low success rate required analyzing parameters like step limits, evaporation rate, and ant count, leading to the conclusion that standard ACO needs far more iterations to perform reliably on random grids. A secondary challenge was ensuring fair memory management, requiring consistent placement of `tracemalloc.start()` at the start of each algorithm to obtain comparable results.